

Лабораторная работа № 1

Объяснение требований и кода

Материал для устной защиты: что реализовано, как работает и чем проверяется.

По текущему коду проекта lab1-template. Подготовлено 3 октября 2026 г.

1. Цель работы

Нужно реализовать HTTP API для сущности Person, хранить данные в базе, написать unit-тесты, упаковать приложение в Docker и автоматизировать сборку, тестирование и deployment. CRUD означает создание, чтение, изменение и удаление данных.

Требование задания	Реализация и состояние
Пять операций REST API	Go + Gin; GET списка и записи, POST, PATCH, DELETE.
JSON и корректные HTTP-статусы	200, 201, 204, 400, 404, 500; POST возвращает Location.
Хранение данных в БД	PostgreSQL, таблица persons, доступ через pgxpool.
4-5 unit-тестов	5 основных функций, 25 подтестов для Handler с Mock Service.
Docker	Multi-stage Dockerfile и локальный Docker Compose.
Сборка и тесты в GitHub Actions	build.yml: unit-тесты, сборка Go и Docker.
Интеграционные тесты	Newman проверяет 5 запросов к API на Railway.
Heroku и deployment через Actions	Не реализовано в исходном виде: используется Railway GitHub autodeploy.
Classroom и автооценка	Исходный workflow ещё требует настройки.

Как корректно описать результат

«Я реализовал API и проверил его unit- и интеграционными тестами. CI работает в GitHub Actions. Приложение опубликовано на Railway. Использование Railway вместо Heroku и текущий способ deployment требуют согласования с преподавателем».

Зелёная задача CI подтверждает конкретные проверки. Она не доказывает выполнение всех условий исходного задания: в README требуется именно Heroku и Docker-deployment через GitHub Actions.

2. Архитектура, модель и маршруты

Путь запроса: HTTP → Handler → Service → Repository → PostgreSQL. Handler отвечает за протокол HTTP, Repository - за SQL. Service связывает слои; в этой небольшой работе он в основном делегирует операции Repository.

Модель запроса: `internal/model/person.go`

```
type PersonRequest struct {
    Name    string `json:"name" binding:"required"`
    Age     *int   `json:"age"`
    Address *string `json:"address"`
    Work    *string `json:"work"`
}
```

json задаёт имя поля в JSON. `binding:"required"` требует непустое имя: без него Handler вернёт 400. Указатели `*int` и `*string` позволяют отличить пропущенное поле от переданного нуля или пустой строки. Модель ответа `Person` дополнительно содержит ID; `omitempty` исключает nil-поля из ответа.

Регистрация маршрутов: `cmd/app/main.go`

```
api.GET("/persons", h.GetAll)
api.GET("/persons/:id", h.GetByID)
api.POST("/persons", h.Create)
api.PATCH("/persons/:id", h.Update)
api.DELETE("/persons/:id", h.Delete)
```

Маршруты входят в группу `router.Group("/api/v1")`, поэтому полный адрес создания - `/api/v1/persons`. `:id` является параметром пути. Gin выбирает Handler по HTTP-методу и адресу.

Service: передача операции в Repository

```
func (s *PersonService) Create(
    ctx context.Context, person model.PersonRequest,
) (int64, error) {
    return s.repo.Create(ctx, person)
}
```

Это сокращённая по форматированию версия текущего метода. `context` передаётся от HTTP-запроса к работе с БД. Отмена запроса может отменить SQL-операцию. Результат содержит ID и ошибку; ошибки затем преобразуются Handler в HTTP-ответ.

Что сказать преподавателю

«Разделение слоёв позволяет менять хранение данных и тестировать HTTP-логику отдельно. Сейчас Service простой, но в него можно добавлять бизнес-правила, не смешивая их с обработкой HTTP и SQL».

3. POST: создание и заголовок Location

Handler: чтение и проверка JSON

```
var req model.PersonRequest
if err := c.ShouldBindJSON(&req); err != nil {
    c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
    return
}
id, err := h.service.Create(c.Request.Context(), req)
```

ShouldBindJSON заполняет структуру и применяет validation. Невалидный JSON, неверный тип age или отсутствие name приводят к 400. При ошибке Service текущий Handler возвращает 500, а при успехе формирует ответ создания.

Repository: INSERT с параметрами

```
err := r.db.QueryRow(
    ctx,
    `INSERT INTO persons (name, age, address, work)
    VALUES ($1, $2, $3, $4)
    RETURNING id`,
    person.Name, person.Age, person.Address, person.Work,
).Scan(&id)
```

Значения передаются отдельно от SQL через \$1-\$4. В коде нет склейки пользовательских значений в текст запроса. RETURNING id получает идентификатор созданной строки, а Scan записывает его в переменную id.

Требование задания: 201 и пустое тело

```
c.Header("Location", "/api/v1/persons/"+
    strconv.FormatInt(id, 10))
c.Status(http.StatusCreated)
```

201 Created означает успешное создание. Location показывает адрес новой записи, например /api/v1/persons/7. c.Status устанавливает статус без JSON-тела. Это важно: по заданию POST должен вернуть пустое тело, а не объект Person.

Как доказать работу

Отправить POST с name, age, address и work. Показать статус 201, Location и длину тела 0. Затем выполнить GET по полученному Location: это подтверждает, что запись действительно доступна из БД. Нельзя предполагать ID = 1.

Для API-демонстрации используйте последовательность команд из практического руководства lab1-defense-ru.pdf.

4. GET, PATCH, DELETE и обработка ошибок

GET: поиск записи и отличие 404 от 500

```
id, err := strconv.ParseInt(c.Param("id"), 10, 64)
// Если ID не число: HTTP 400.
person, err := h.service.GetByID(c.Request.Context(), id)
if err != nil {
    if errors.Is(err, pgx.ErrNoRows) {
        c.JSON(http.StatusNotFound,
            gin.H{"message": "person not found"})
    } else {
        c.JSON(http.StatusInternalServerError,
            gin.H{"message": "internal server error"})
    }
    return
}
c.JSON(http.StatusOK, person)
```

Это фрагмент с сокращённой веткой проверки ID. `pgx.ErrNoRows` означает отсутствие записи, поэтому возвращается 404. Другая ошибка, например отказ БД, означает 500. Для списка используется `rows.Next()`, а пустой результат возвращается как `[]` благодаря `make([]model.Person, 0)`.

PATCH: сохранить поля, которые не переданы

```
UPDATE persons
SET name = $1, age = COALESCE($2, age),
    address = COALESCE($3, address),
    work = COALESCE($4, work)
WHERE id = $5
RETURNING id, name, age, address, work
```

Пропущенные `age/address/work` превращаются в `nil`, то есть SQL `NULL`. `COALESCE` выбирает переданное значение, а при `NULL` сохраняет старое. `RETURNING` позволяет вернуть обновлённый объект с 200. В текущем API `name` обязательно и для `PATCH`. Переданные 0 или пустая строка учитываются; очистить поле до SQL `NULL` этим запросом нельзя.

DELETE: проверить, что строка существовала

```
// После DELETE FROM persons WHERE id = $1:
if result.RowsAffected() == 0 {
    return pgx.ErrNoRows
}
// Handler при успешном удалении:
c.Status(http.StatusNoContent)
```

Ноль изменённых строк превращается в 404 через `Handler`. Успешное удаление возвращает 204 без тела. На защите покажите `GET` после `DELETE`: он должен вернуть 404.

5. PostgreSQL и переменные окружения

Настройка подключения в main.go

```
databaseURL := os.Getenv("DATABASE_URL")
if databaseURL == "" {
    databaseURL = "postgres://program:test@localhost:5432/persons"
}
port := os.Getenv("PORT")
if port == "" {
    port = "8080"
}
db, err := pgxpool.New(ctx, databaseURL)
// После проверки ошибки и Ping:
if err := router.Run(":" + port); err != nil {
    log.Fatal(err)
}
```

Фрагмент объединяет настройку в начале main.go и запуск в конце; между ними создаются Repository, Service, Handler и маршруты. pgxpool - пул подключений. Ping проверяет реальную доступность БД, поскольку создание пула само по себе не доказывает успешное соединение. Для Ping установлен timeout 10 секунд.

Инициализация таблицы при старте

```
CREATE TABLE IF NOT EXISTS persons (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    age INTEGER,
    address VARCHAR(255),
    work VARCHAR(255)
)
```

main.go выполняет этот SQL с timeout 30 секунд до запуска HTTP-сервера. BIGSERIAL формирует ID, PRIMARY KEY задаёт ключ, NOT NULL запрещает отсутствие name. IF NOT EXISTS позволяет повторный старт без удаления таблицы. Для учебного приложения это простой способ подготовки БД; это не полноценная система миграций.

Почему нельзя всегда использовать localhost

При локальном Go-процессе localhost указывает на компьютер. В контейнере приложения это адрес самого контейнера. Поэтому в Compose используется postgres:5432, а в Railway - ссылка на DATABASE_URL сервиса Postgres.

```
# В Variables сервиса приложения на Railway:
Name: DATABASE_URL
Value: ${Postgres.DATABASE_URL}
```

Если переменная отсутствует, приложение использует локальный fallback и в Railway получает connection refused. PUBLIC domain направляет запросы на порт приложения; Target Port должен совпадать с PORT. Сам URL с паролем не нужно показывать преподавателю.

6. Dockerfile и Docker Compose

Dockerfile: две стадии сборки

```
FROM golang:1.25-alpine AS builder

WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o server ./cmd/app

FROM alpine:3.22

WORKDIR /app
COPY --from=builder /app/server .

EXPOSE 8080
CMD ["/server"]
```

1. builder содержит компилятор Go. Сначала копируются go.mod и go.sum и загружаются зависимости. Такой порядок позволяет Docker повторно использовать слой зависимостей, если они не изменились.
2. CGO_ENABLED=0 отключает зависимость от C-компонентов, GOOS=linux задаёт целевую ОС. Результат - server.
3. В финальную Alpine-стадию копируется только binary. Go-компилятор в рабочем образе не нужен.
4. CMD запускает приложение. EXPOSE документирует порт; он не публикует порт на компьютере сам по себе.

Compose: приложение и PostgreSQL вместе

```
services:
  app:
    build: .
    environment:
      DATABASE_URL: postgres://program:test@postgres:5432/persons
      PORT: "8080"
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
```

Это фрагмент текущего Compose. Имя сервиса postgres доступно как DNS-имя в сети Compose. healthcheck использует pg_isready, а service_healthy откладывает запуск app до готовности БД. Том db-data сохраняет данные после остановки контейнеров.

```
docker compose up -d --build
docker compose ps
docker compose logs --tail 50 app
docker compose down
```

Покажите работающие контейнеры и доступность API на localhost:8080. Обычный down сохраняет том; down -v удаляет его.

7. Unit-тесты: интерфейс, Mock и httptest

Зависимость Handler задаётся интерфейсом

```
type personService interface {
    Create(context.Context, model.PersonRequest) (int64, error)
    GetByID(context.Context, int64) (*model.Person, error)
    GetAll(context.Context) ([]model.Person, error)
    Update(context.Context, int64, model.PersonRequest) (*model.Person, error)
    Delete(context.Context, int64) error
}

type PersonHandler struct {
    service personService
}

func NewPersonHandler(service personService) *PersonHandler {
    return &PersonHandler{
        service: service,
    }
}
```

Handler принимает любой объект, реализующий методы интерфейса. В приложении передаётся настоящий *service.PersonService; в тесте - mockPersonService. Поэтому unit-тест Handler не запускает Repository и PostgreSQL.

Mock возвращает предсказуемый результат

```
svc := &mockPersonService{
    create: func(ctx context.Context,
        req model.PersonRequest) (int64, error) {
        return 1, nil
    },
}
```

Это сокращённый пример настройки mock из TestCreatePerson. Полный тест также проверяет аргументы, переданные Service. Строка var _ personService = (*mockPersonService)(nil) проверяет соответствие интерфейсу во время компиляции.

Запрос выполняется без сетевого сервера

```
req := httptest.NewRequest(method, path, strings.NewReader(body))
req.Header.Set("Content-Type", "application/json")
w := httptest.NewRecorder()
router.ServeHTTP(w, req)
```

httptest.NewRequest создаёт HTTP-запрос; NewRecorder сохраняет ответ; ServeHTTP пропускает запрос через Gin и Handler. Для POST проверяются 201, Location и пустое тело. Для других операций проверяются JSON, 400, 404 и 500, в том числе обёрнутый pgx.ErrNoRows.

```
go test ./...
go test -v ./internal/handler
go test ./internal/handler -list '^Test'
```

В проекте 5 основных тестовых функций и 25 подтестов. Unit-тесты доказывают поведение HTTP-слоя при заданном результате Service. Настоящие SQL-запросы и БД проверяются отдельно интеграционными запросами Newman.

8. CI, Newman и оставшиеся требования

GitHub Actions: автоматическая проверка кода

```
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

# Основные команды задачи test:
go mod download
go test ./...
go build ./cmd/app
docker build -t lab1-ci-cd .
```

Блок показывает триггеры и команды из build.yml; это не полный готовый YAML. На временной Ubuntu-машине checkout получает исходники, setup-go устанавливает Go, затем выполняются проверки. Ошибка команды завершает задачу с failure. Этот job проверяет Docker-сборку, но не запускает приложение с PostgreSQL.

Наша цепочка deployment и интеграционных тестов

Push в master → CI проверяет код; отдельно Railway получает исходники, собирает Dockerfile и запускает приложение. После события deployment_status со state = success для upbeat-delight / production GitHub Actions запускает Newman. В этом проекте deployment выполняет Railway, а не команда GitHub Actions.

```
newman run "postman/[inst] Lab1.postman_collection.json" \
  --environment "postman/[inst][railway] Lab1.postman_environment.json" \
  --delay-request 100 --timeout-request 30000 \
  --reporters cli,junit --reporter-junit-export newman/results.xml
```

Это команда для Bash в CI. Перед ней workflow проверяет доступность API через curl. Newman выполняет POST, GET по ID, GET списка, PATCH и DELETE на настоящем сервере. Ожидается 5 запросов, 5 именованных проверок и 0 ошибок. Отчёт сохраняется как artifact railway-newman-results. Выполняется тестирование именно опубликованной версии.

Что показать в интерфейсе Actions

Покажите успешную задачу test для push, затем отдельную Newman tests on Railway после deployment. Skipped является ожидаемым, если задача не предназначена для данного события. Возможен ручной запуск через Actions → CI → Run workflow.

Почему Classroom остаётся красным

classroom.yml из шаблона использует Heroku environment с пустым baseUrl. Поэтому шаг Run API Tests не имеет адреса приложения. Следующие шаги автооценки и записи в таблицу зависят от настроек преподавателя, включая GOOGLE_API_KEY, и не подтверждены. Не следует объявлять все checks успешными.

Главное ограничение по тексту задания

Исходное задание требует Heroku и deployment с Docker через GitHub Actions. Сейчас работает Railway GitHub autodeploy. Нужно получить согласование преподавателя на замену или реализовать исходную схему. CRUD, PostgreSQL, unit-тесты, Docker, CI и Newman реализованы; это не отменяет оставшиеся требования.